

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS

COURSE CODE: CSA0403

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

Scenario based Questions

Code of Conduct:

I, **Ashira Bridget Susan S (192524316)**, certify that this submission is my original work and that

I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Ashira Bridget Susan S

Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

A railway ticket booking system experiences heavy load during festival seasons. The system handles three categories of processes:

- P1 – Payment gateway transactions (High priority)
- P2 – Seat availability checks (Medium priority)
- P3 – Ticket confirmation and printing (Low priority)
- Burst times vary from 2 ms to 20 ms.

Recently, customers complained that ticket confirmations are taking too long.

Tasks

1. Identify the most suitable scheduling algorithm (FCFS, SJF, Priority, RR) for this scenario.
2. Justify why your chosen method improves CPU utilization and reduces customer wait time.
3. Provide a sample execution order for the above categories.
4. Discuss one drawback of this scheduling method in real-time systems.

Solution:-

1. Most Suitable Scheduling Algorithm

Priority Scheduling

Since the railway ticket booking system has processes with different importance levels, Priority Scheduling is the most suitable.

Process	Task	Priority
P1	Payment Gateway Transactions	High
P2	Seat Availability Checks	Medium
P3	Ticket Confirmation and Printing	Low

2. Justification

- Payment transactions must be completed quickly to avoid payment failures.
- Priority Scheduling allocates the CPU to the most important tasks first.

- It improves customer experience by ensuring critical operations are processed immediately.
- CPU utilization remains high because high-priority tasks are executed without unnecessary delays.
- Medium-priority tasks are handled after payments, while less critical tasks are executed later.

Benefits:

- Faster response for critical tasks.
- Reduced waiting time for payment processing.
- Better resource allocation during heavy festival loads.

3. Sample Execution Order

Assume the following burst times:

Process	Burst Time	Priority
P1	5 ms	High
P2	10 ms	Medium
P3	15 ms	Low

Execution Order

Arrival Time = 0

P1 (5 ms) → P2 (10 ms) → P3 (15 ms)

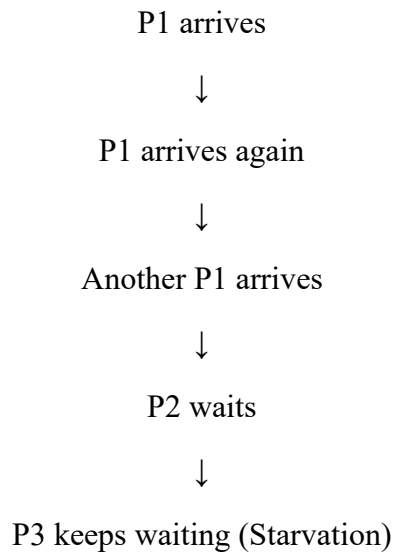
0 5 15 30
 |---P1---|----P2----|-----P3-----|

If multiple payment transactions arrive, they will be executed before medium and low-priority tasks.

4. Drawback of Priority Scheduling

The major drawback is **Starvation**.

- Low-priority processes (such as ticket confirmation and printing) may wait indefinitely if high-priority processes continue arriving.
- During festival seasons, numerous payment requests may prevent ticket confirmations from being processed promptly.

Example:**Possible Solution:**

Using **Aging**, where the priority of waiting processes gradually increases over time, can reduce starvation.

Conclusion

- **Scheduling Algorithm:** Priority Scheduling
 - **Reason:** Ensures critical payment transactions are processed first during heavy loads.
 - **Execution Order:** $P1 \rightarrow P2 \rightarrow P3$.
 - **Advantage:** Faster response time and better CPU utilization for important tasks.
 - **Drawback:** Starvation of low-priority processes, which can be mitigated using aging.
-

Question 2 — Deadlock in a Banking Transaction Server

A bank server handles the following operations simultaneously:

- T1: Transfers money between accounts
- T2: Updates passbook entries
- T3: Generates monthly statements

Due to simultaneous locking of Account Database (A), Customer Info Database (B), and Transaction Logs (C), the system enters a state where:

- T1 holds A and requests B
- T2 holds B and requests C
- T3 holds C and requests A

Tasks

1. Draw the resource request situation (text-based RAG).
2. Identify whether a deadlock has occurred and justify your answer.
3. Recommend one specific technique (avoidance/prevention/recovery) and explain how it fixes the issue.
4. Suggest a redesign for the locking order to avoid future deadlocks.

Solution:-

1. Resource Allocation Graph (Text-Based)

T1 holds A and requests B



T2 holds B and requests C



T3 holds C and requests A



Combined representation:

$A \rightarrow T1 \rightarrow B$
 $B \rightarrow T2 \rightarrow C$
 $C \rightarrow T3 \rightarrow A$

where,

- $A \rightarrow T1$: Resource A is allocated to T1.
- $T1 \rightarrow B$: T1 is requesting Resource B.
- $B \rightarrow T2$: Resource B is allocated to T2.
- $T2 \rightarrow C$: T2 is requesting Resource C.

- $C \rightarrow T3$: Resource C is allocated to T3.
- $T3 \rightarrow A$: T3 is requesting Resource A.

This forms a cycle:

$$T1 \rightarrow B \rightarrow T2 \rightarrow C \rightarrow T3 \rightarrow A \rightarrow T1$$

2. Has a Deadlock Occurred?

Yes, a deadlock has occurred.

Justification

All four deadlock conditions are satisfied:

1. Mutual Exclusion

- Resources A, B, and C can be used by only one transaction at a time.

2. Hold and Wait

- T1 holds A and waits for B.
- T2 holds B and waits for C.
- T3 holds C and waits for A.

3. No Preemption

- Resources cannot be forcibly taken away from a transaction.

4. Circular Wait

- A circular dependency exists:

$$T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$$

Since all four conditions are true, the system is in a deadlock state.

3. Recommended Technique

Deadlock Prevention (Resource Ordering Method)

Assign a fixed order for acquiring resources:

$$A \rightarrow B \rightarrow C$$

Every transaction must request resources only in this order.

Example

Instead of:

$T1 : A \rightarrow B$

$T2 : B \rightarrow C$

$T3 : C \rightarrow A$

use:

$T1 : A \rightarrow B$

$T2 : A \rightarrow B \rightarrow C$

$T3 : A \rightarrow C$

No transaction is allowed to request:

$C \rightarrow A$

or

$B \rightarrow A$

This removes the possibility of circular wait.

How It Fixes the Issue

- Circular waiting is eliminated.
- Transactions either obtain resources in the proper order or wait safely.
- Deadlocks cannot occur because one of the necessary deadlock conditions is removed.

4. Suggested Redesign

Implement a **Global Locking Order Policy**.

Resource Priority:

1. Account Database (A)

↓

2. Customer Database (B)

↓

3. Transaction Logs (C)

Every transaction must follow:

$A \rightarrow B \rightarrow C$

Example

Transaction	Locking Order
T1	$A \rightarrow B$
T2	$A \rightarrow B \rightarrow C$
T3	$A \rightarrow C$

Transactions requesting resources in any other order should wait until they can obtain them correctly.

Advantages

- Prevents circular waiting.
- Eliminates deadlocks.
- Improves system reliability during concurrent banking operations.
- Ensures consistent resource allocation.

Conclusion

- **Deadlock Status:** Yes, deadlock has occurred.
 - **Reason:** Circular wait exists among T1, T2, and T3.
 - **Recommended Technique:** Deadlock Prevention using Resource Ordering.
 - **Redesign:** Enforce a global locking order ($A \rightarrow B \rightarrow C$) for all transactions to avoid future deadlocks.
-

Question 3 — Mobile OS App Switching Lag

A mobile operating system is designed to support smooth app switching.

But users report that:

- **Opening WhatsApp delays the background music player.**
- **Switching between social media apps feels laggy.**
- **CPU shows high idle times despite multiple running apps.**

The OS currently uses Round Robin (quantum = 20 ms) with 20 background apps active.

Tasks

- 1. Analyze why the CPU remains idle despite many active processes.**
- 2. Determine whether the time quantum is helping or hurting performance.**
- 3. Recommend a better scheduling method or quantum value, with justification.**
- 4. Explain how the change will reduce switching lag and improve system throughput.**

Solution:-

1. Why does the CPU remain idle despite many active processes?

The CPU may remain idle because:

- Most background apps are waiting for I/O operations (network access, notifications, storage access, etc.).
- Frequent context switching among 20 applications increases scheduling overhead.
- The fixed time quantum of 20 ms may not suit both foreground and background tasks.
- The scheduler spends time switching between processes instead of executing useful work.

Therefore, even with many active processes, the CPU is not always performing productive computations.

2. Is the Time Quantum Helping or Hurting Performance?

The 20 ms time quantum is hurting performance.

Reasons:

- With 20 active applications, each app must wait longer before receiving CPU time again.
- Foreground applications such as WhatsApp require faster responses.

- Background applications unnecessarily compete with interactive tasks.
- Users experience delays while switching between applications.

Example

$$20 \text{ Applications} \times 20 \text{ ms} = 400 \text{ ms}$$

Worst-case waiting time $\approx 400 \text{ ms}$

A foreground application may have to wait hundreds of milliseconds before running again, causing noticeable lag.

3. Recommended Scheduling Method

Priority-Based Round Robin (Multilevel Feedback Queue) with a smaller time quantum (10 ms).

Suggested Design

Process Type	Scheduling
Foreground Apps	Higher Priority
Background Apps	Lower Priority
Interactive Apps	10 ms Quantum
Background Tasks	Larger Quantum if required

Examples:

- WhatsApp
- Phone calls
- Music player
- Notifications

These should receive higher priority than background applications.

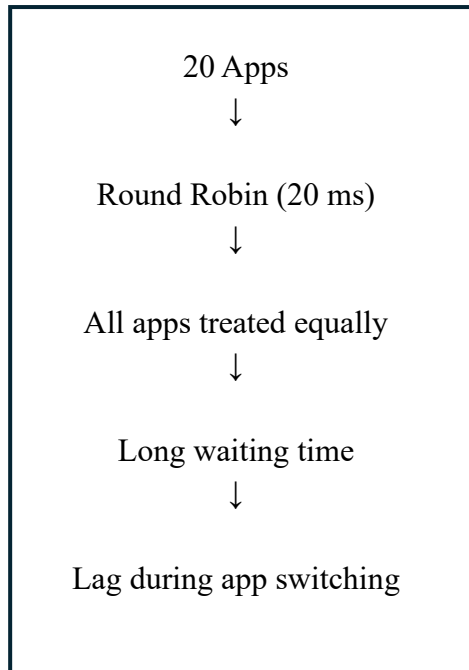
Why?

- Interactive applications receive CPU time more quickly.
- Background processes do not unnecessarily delay user actions.
- Smaller quantum improves responsiveness for app switching.

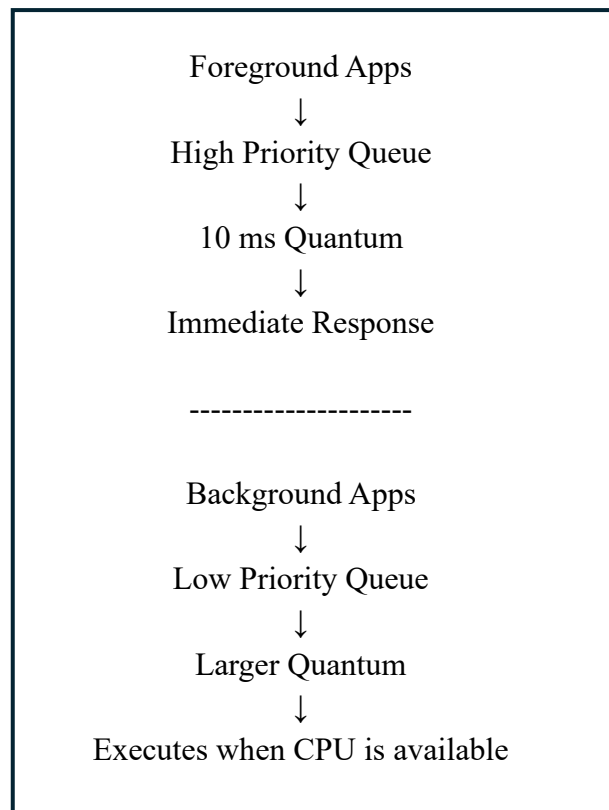
4. How Will This Improve Performance?

Reduction in Switching Lag

Before:



After:



Improvements

1. Faster app switching.
2. Better responsiveness for WhatsApp and social media apps.
3. Reduced waiting time for interactive applications.
4. Improved CPU utilization.
5. Increased system throughput because CPU time is allocated more efficiently.
6. Background music playback remains smooth while foreground applications run.

Conclusion

- **Reason for Idle CPU:** Frequent waiting for I/O operations and inefficient scheduling among many background apps.
 - **Current Quantum:** 20 ms negatively affects responsiveness when many processes are active.
 - **Recommended Solution:** Use **Priority-Based Round Robin (Multilevel Feedback Queue)** with a **10 ms quantum** for foreground tasks.
 - **Benefits:** Reduced app-switching lag, improved CPU utilization, smoother multitasking, and better overall system throughput.
-

Question 4 — Deadlock Risk in an Operating System Update

During a Windows OS update, several modules run in parallel:

- **M1: Copy update files**
- **M2: Install drivers**
- **M3: Update registry**
- **M4: Validate system integrity**

Resources involved:

- **R1 = Disk I/O**
- **R2 = Kernel lock**
- **R3 = Configuration lock**

Recently logs show:

- **M1 holds R1, waiting for R2**
- **M2 holds R2, waiting for R3**
- **M3 holds R3, waiting for R1**

Tasks

- 1. Identify whether this sequence can lead to a deadlock.**
- 2. If yes, explain the exact circular wait condition.**
- 3. Propose Banker's algorithm style resource allocation to avoid risk.**
- 4. Suggest how OS designers can prioritize modules to improve CPU utilization during update.**

Solution:-

- 1. Can this sequence lead to a deadlock?**

Yes, this sequence can lead to a deadlock.

The modules are holding one resource while waiting for another resource.

Module	Holds	Waiting For
M1	R1 (Disk I/O)	R2 (Kernel Lock)
M2	R2 (Kernel Lock)	R3 (Configuration Lock)
M3	R3 (Configuration Lock)	R1 (Disk I/O)

Since every module is waiting for a resource held by another module, the system can enter a deadlock state.

2. Explain the Circular Wait Condition

The circular dependency is:

```

M1 holds R1
  ↓
waits for R2
  ↓
M2 holds R2
  ↓
waits for R3
  ↓
M3 holds R3
  ↓
waits for R1
  ↓
M1

```

Or,

$$R1 \rightarrow M1 \rightarrow R2 \rightarrow M2 \rightarrow R3 \rightarrow M3 \rightarrow R1$$

This forms a cycle.

All four deadlock conditions are present:

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

Hence, deadlock is possible.

3. Banker's Algorithm Style Resource Allocation

Banker's Algorithm allocates resources only when the system remains in a **safe state**.

Safe Allocation Example

Available Resources:

R1 = Free

R2 = Free

R3 = Free

Execute modules sequentially:

Step 1:

Allocate:

M1 → R1 and R2

M1 completes
↓
Release R1 and R2

Step 2:

Allocate:

M2 → R2 and R3

M2 completes
↓
Release R2 and R3

Step 3:

Allocate:

M3 → R1 and R3

M3 completes
↓
Release R1 and R3

Step 4:

Execute:

M4 (Validation Process)

M4 completes.

Safe Sequence

M1 → M2 → M3 → M4

Before allocating resources, the OS checks whether granting the request keeps the system in a safe state. If not, the request is delayed until resources become available.

This prevents deadlock from occurring.

4. Prioritizing Modules to Improve CPU Utilization

The operating system can use **Priority Scheduling** during updates.

Module	Priority
M1 – Copy Files	High
M2 – Install Drivers	High
M3 – Update Registry	Medium
M4 – System Validation	Low

Suggested Execution Order

M1
↓
M2
↓
M3
↓
M4

Benefits

- Critical update operations finish first.
- CPU resources are efficiently utilized.
- Validation tasks run after essential components are installed.

- Reduced waiting time for important modules.
 - Better overall system performance during updates.
-

Conclusion

- **Deadlock Possible?** Yes.
 - **Reason:** Circular wait exists among M1, M2, and M3.
 - **Solution:** Apply Banker's Algorithm to ensure resource allocation always remains in a safe state.
 - **CPU Utilization Improvement:** Use Priority Scheduling so that important update modules execute first, resulting in smoother and more efficient OS updates.
-

Question 5 — Cloud Server CPU Utilization Analysis

A cloud hosting provider notices:

- 80% of the time, VM processes wait for network or disk
- Only 20% of the time is spent on CPU
- Each server runs 7 VMs

Using the formula:

where p = I/O wait probability

Tasks

1. Calculate CPU utilization.
2. Interpret whether the server is underutilized or optimally loaded.
3. Suggest how increasing or decreasing number of VMs affects throughput.
4. Propose two OS-level strategies for improving CPU utilization in this cloud environment.

Solution:-

The given formula is:

$$\text{CPU Utilization} = 1 - p^n$$

where:

- p = Probability that a process is waiting for I/O
- n = Number of processes (VMs)

Given:

- $p = 0.8$
 - $n = 7$
-

1. Calculate CPU Utilization

Calculation

- $0.8^7 = 0.2097152$
- CPU Utilization = $1 - 0.2097152$
- CPU Utilization = **0.7902848**

Percentage

CPU Utilization $\approx 79.03\%$

2. Is the Server Underutilized or Optimally Loaded?

- CPU utilization is approximately **79%**.
- The CPU is busy most of the time.
- About **21%** of the time, all VMs are simultaneously waiting for I/O operations.

Interpretation

- The server is **reasonably well utilized**.
- It is not severely underutilized.
- There is still room for improvement since the CPU remains idle about 21% of the time.

Conclusion: The server is fairly well utilized but not operating at maximum efficiency.

3. Effect of Changing the Number of VMs

Increasing the Number of VMs

Suppose we increase to 10 VMs.

- CPU Utilization = $1 - 0.8^{10}$
- $= 1 - 0.107374$
- $= 89.26\%$

Effects

- Higher CPU utilization.
- Better throughput.
- Less idle CPU time.
- More tasks can execute while others wait for I/O.

Decreasing the Number of VMs

Suppose only 3 VMs are running.

- CPU Utilization = $1 - 0.8^3$
- = $1 - 0.512$
- = 48.8%

Effects

- Lower throughput.
- Increased CPU idle time.
- Resources are not fully utilized.

Increasing the number of VMs generally improves throughput up to the point where excessive context switching and resource contention begin to degrade performance.

4. OS-Level Strategies to Improve CPU Utilization

Strategy 1: Increase Multiprogramming Level

- Allow more VMs or processes to run simultaneously.
- When one VM waits for disk or network operations, another can utilize the CPU.
- This reduces CPU idle time.

Benefit:

- Improved throughput.
- Better resource utilization.

Strategy 2: Use Efficient CPU Scheduling

The operating system can use:

- Multilevel Feedback Queue Scheduling

- Priority Scheduling
- Optimized Round Robin scheduling

These methods:

- Reduce waiting time.
- Improve responsiveness.
- Ensure the CPU is allocated efficiently among competing VMs.

Benefit:

- Higher CPU utilization.
- Better overall system performance.

Conclusion

Parameter	Value
I/O Wait Probability (p)	0.8
Number of VMs (n)	7
CPU Utilization	79.03%
Server Status	Fairly well utilized
Increasing VMs	Improves throughput and utilization
Decreasing VMs	Reduces throughput and increases idle time
Improvements	Increase multiprogramming level and use efficient CPU scheduling

Final Answer: The cloud server achieves approximately **79.03% CPU utilization**. Increasing the number of VMs can improve throughput and utilization (within practical limits), while OS-level techniques such as higher multiprogramming and efficient CPU scheduling can further enhance performance.

Question 6 – CPU Scheduling Scenario

Scenario:

A large-scale CPU scheduling environment in organization 1 experiences performance and reliability issues during peak usage. Multiple processes/resources interact simultaneously,

causing delays, contention, and reduced throughput.

1. Analyze the root cause of the observed behavior.
2. Determine the most appropriate Operating System technique/algorithm to address the issue.
3. Justify your recommendation with respect to CPU utilization, throughput, response time, or fairness.
4. Design an improved system configuration or execution sequence for the given scenario.
5. Evaluate one limitation or trade-off of your proposed solution.

Solution:-

1. Root Cause of the Problem

The performance issues arise due to:

- Heavy CPU contention among multiple processes.
- Improper scheduling during peak usage.
- Increased waiting time and frequent context switching.
- Unfair CPU allocation causing some processes to experience delays.
- Reduced throughput when several processes compete for resources simultaneously.

Root Cause: Inefficient CPU scheduling and resource contention during peak load conditions.

2. Recommended Operating System Technique

Multilevel Feedback Queue (MLFQ) Scheduling

MLFQ is suitable because:

- It supports both interactive and CPU-intensive processes.
- It dynamically changes process priorities based on their behavior.
- Short and interactive tasks receive faster CPU access.

- Long-running processes are gradually moved to lower-priority queues.

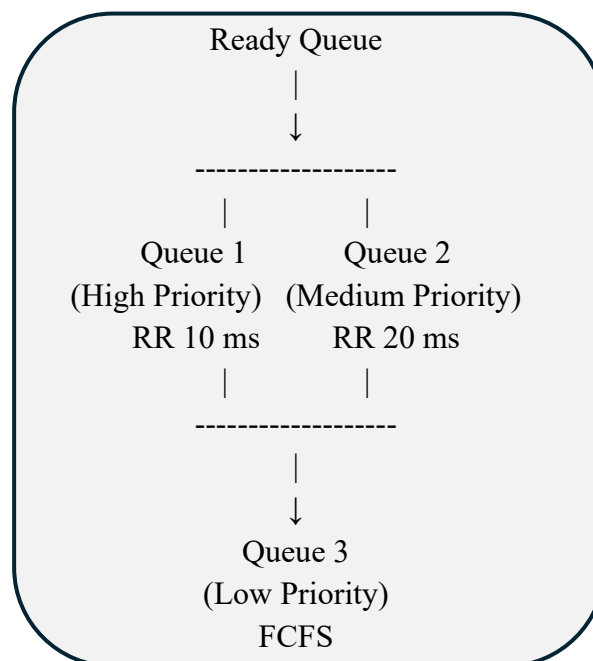
3. Justification

Parameter	Improvement
CPU Utilization	High
Throughput	Improved
Response Time	Faster for interactive processes
Fairness	Better distribution of CPU time
Starvation Prevention	Possible through aging techniques

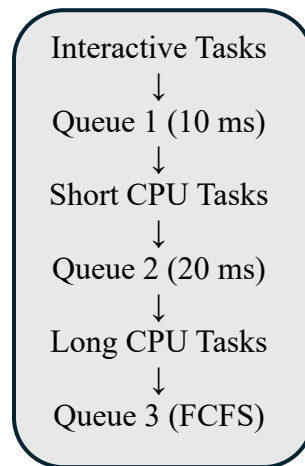
Why MLFQ?

- Processes requiring immediate attention are executed first.
- Background processes continue execution without completely blocking others.
- CPU resources are utilized efficiently during peak loads.
- Provides a balance between responsiveness and fairness.

4. Improved System Configuration - Queue Structure



Execution Sequence



Examples:

- User requests
- Network services
- Background processing
- Report generation tasks

Working

1. New processes enter Queue 1.
2. If unfinished, they move to Queue 2.
3. Long-running processes are shifted to Queue 3.
4. Aging can promote waiting processes back to higher-priority queues.

This improves:

- Response time.
- CPU utilization.
- Overall throughput.

5. Limitation / Trade-Off

Limitation: Increased Scheduling Complexity

- MLFQ is more complex than FCFS or Round Robin.
- Selecting appropriate time quantum values requires careful tuning.
- Improper queue configuration may still lead to starvation of lower-priority processes.
- Additional overhead is introduced when processes are frequently moved between queues.

Conclusion

- **Root Cause:** CPU contention and inefficient scheduling during peak loads.
- **Recommended Technique:** Multilevel Feedback Queue (MLFQ) Scheduling.
- **Benefits:** Improved CPU utilization, throughput, fairness, and response time.
- **Improved Design:** Multiple priority queues with different scheduling policies and aging mechanisms.
- **Trade-Off:** Higher implementation complexity and scheduling overhead.

Final Answer: Implementing **MLFQ Scheduling** with appropriate time quanta and aging mechanisms provides an effective balance between performance, fairness, and responsiveness in large-scale CPU scheduling environments.

Question 7:

Scenario:

A leading international e-commerce company operates across 70 countries and processes nearly 10 million customer requests every day. The system consists of multiple activities such as user authentication, product search, shopping cart updates, payment processing, inventory management, and shipment tracking. During festive sales, the response time of the application increases significantly due to the large number of simultaneously executing tasks. The Chief Technology Officer (CTO) has formed a task force to investigate how the operating system manages these activities as processes and how process state transitions affect overall system performance. The team is also interested in understanding the role of context switching and Process Control Blocks (PCB) in maintaining efficient execution.

Questions:

1. Analyze the various processes involved in the given scenario.
2. Differentiate between a program and a process with suitable examples from the case study.
3. Explain how Process Control Blocks (PCB) are utilized by the operating system.
4. Illustrate the different process states that a payment transaction may undergo.
5. Evaluate the impact of context switching during peak sale periods.
6. Predict the consequences of excessive process creation in the system.

7. Design an efficient process hierarchy for the organization.
8. Justify the need for multiprogramming in this environment.
9. Recommend strategies to improve process management and scalability.
10. Develop a process lifecycle model suitable for the e-commerce platform.

Solution:-

1. Analyze the Various Processes Involved

The e-commerce platform performs multiple processes simultaneously:

Process	Function
User Authentication	Verifies user credentials
Product Search	Retrieves product details
Shopping Cart Management	Adds/removes items from cart
Payment Processing	Handles online payments
Inventory Management	Updates stock availability
Shipment Tracking	Tracks delivery status
Notification Service	Sends SMS/Email updates
Database Operations	Stores and retrieves information

All these processes execute concurrently to serve millions of customer requests efficiently.

2. Differentiate Between Program and Process

Program	Process
A set of instructions stored on disk	A program currently executing in memory
Passive entity	Active entity
Does not consume CPU	Uses CPU and memory resources
Example: Payment module code	A customer's payment transaction currently executing

Example

- **Program:** PaymentService.java
- **Process:** Payment request made by Customer ID 12345 during checkout.

Thus, one program can create multiple processes simultaneously.

3. Role of Process Control Block (PCB)

PCB stores information required by the operating system to manage processes.

It contains:

- Process ID (PID)
- Process state
- Program counter
- CPU registers
- Scheduling information
- Memory allocation details
- I/O status information

Example

PCB

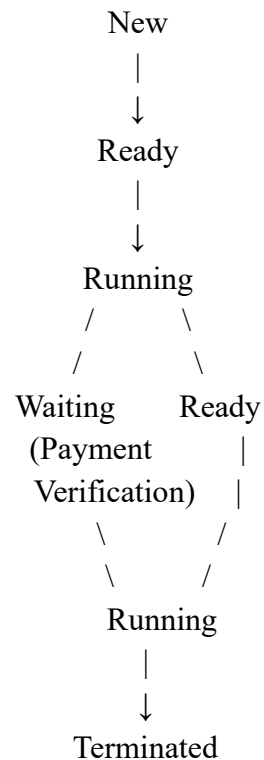
```
-----  
PID = 4501  
-----  
State = Running  
-----  
Priority = High  
-----  
CPU Registers  
-----  
Program Counter  
-----  
Memory Details  
-----  
I/O Information  
-----
```

The OS uses PCB during:

- Scheduling

- Context switching
 - Process creation
 - Process termination
-

4. Process States of a Payment Transaction



Explanation

1. **New**
 - Customer initiates payment.
2. **Ready**
 - Waiting for CPU allocation.
3. **Running**
 - Payment processing begins.
4. **Waiting**
 - Waiting for bank verification or OTP confirmation.
5. **Running**
 - Payment confirmation is completed.
6. **Terminated**

- Transaction successfully finishes.
-

5. Impact of Context Switching During Peak Sales

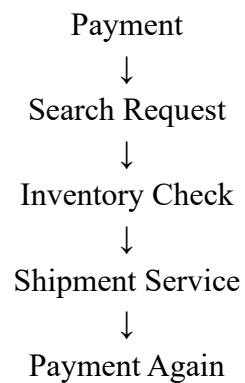
Advantages

- Allows multiple customer requests to share CPU resources.
- Improves responsiveness.

Disadvantages

- Excessive context switching increases CPU overhead.
- Reduces throughput.
- Increases response time during festive sales.

Example

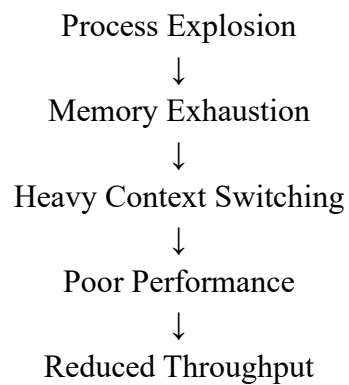


Frequent switching among millions of processes consumes valuable CPU time.

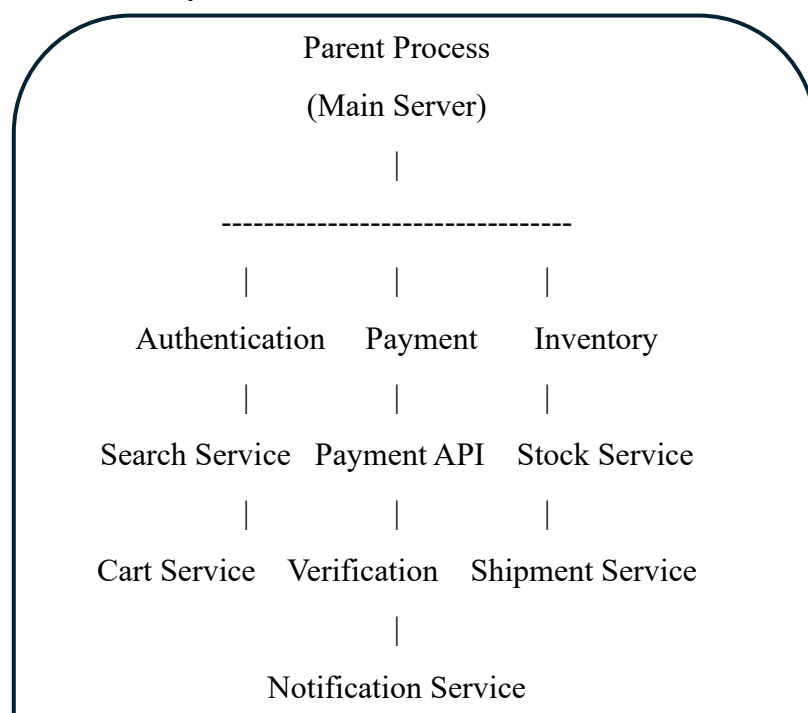
6. Consequences of Excessive Process Creation

- Increased memory consumption.
- Higher scheduling overhead.
- Reduced CPU utilization.
- Increased waiting time.
- Resource contention.
- System slowdown during peak sales.

Extreme cases may result in:



7. Efficient Process Hierarchy



This hierarchical design improves:

- Process management
- Resource allocation
- Scalability

8. Need for Multiprogramming

Multiprogramming allows several processes to execute concurrently.

Benefits

- Higher CPU utilization.

- Improved throughput.
- Reduced CPU idle time.
- Faster response for customers.

Example

While:

- Payment waits for bank verification,
- Product search can utilize the CPU,
- Shipment services can execute simultaneously.

Thus, CPU resources are utilized efficiently.

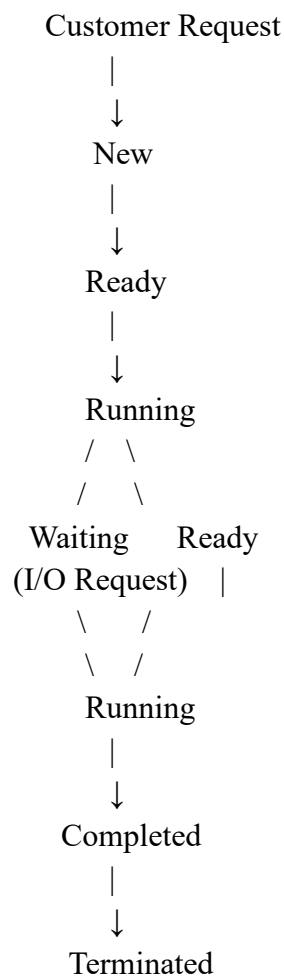
9. Strategies for Improving Process Management

1. Use Multilevel Feedback Queue Scheduling.
2. Employ thread pools instead of creating excessive processes.
3. Implement load balancing across servers.
4. Limit unnecessary process creation.
5. Use asynchronous I/O operations.
6. Apply process prioritization for payment services.
7. Utilize cloud auto-scaling mechanisms.

These techniques improve:

- Performance
 - Scalability
 - CPU utilization
 - Reliability
-

10. Process Lifecycle Model



Working

- Requests are created.
- CPU scheduling allocates execution time.
- I/O operations move processes to the waiting state.
- Completed processes release resources and terminate.

Conclusion

The e-commerce platform relies heavily on efficient process management to handle millions of daily requests. Process Control Blocks, context switching, multiprogramming, and appropriate scheduling mechanisms collectively ensure responsiveness and scalability. Designing an organized process hierarchy and adopting scalable process management strategies significantly improves system performance, particularly during peak festive sales.

Question 8: Process Scheduling

Scenario:

A smart hospital has implemented an automated healthcare management system to handle patient registration, diagnostic services, medical record access, pharmacy management, and emergency response operations. During emergency situations, doctors have reported delays in retrieving patient records because CPU resources are being shared among multiple services. The hospital administration has requested the IT team to investigate different CPU scheduling policies and recommend a suitable scheduling strategy that prioritizes emergency services without starving routine tasks.

Questions:

1. Identify the scheduling challenges present in the hospital management system.
2. Compare FCFS, SJF, Priority Scheduling, and Round Robin for this application.
3. Analyze the impact of scheduling policies on emergency patient care.
4. Determine the most appropriate scheduling algorithm for the given scenario.
5. Evaluate the possibility of starvation and suggest remedies.
6. Recommend a suitable time quantum if Round Robin is selected.
7. Predict system performance during a mass casualty incident.
8. Justify your choice of scheduling algorithm using performance metrics.
9. Propose modifications to improve fairness and responsiveness.
10. Design a scheduling framework for the hospital's future expansion plans.

Solution:-

1. Scheduling Challenges in the Hospital Management System

The major challenges are:

- Emergency services require immediate CPU access.
 - Multiple services compete for CPU resources simultaneously.
 - Delays in retrieving patient records can affect treatment.
 - Maintaining fairness between emergency and routine tasks.
 - Avoiding starvation of low-priority processes.
 - Handling sudden increases in workload during emergencies.
-

2. Comparison of Scheduling Algorithms

Algorithm	Advantages	Disadvantages
FCFS	Simple to implement	High waiting time
SJF	Reduces average waiting time	Difficult to predict burst time
Priority Scheduling	Gives preference to emergency tasks	May cause starvation
Round Robin	Fair CPU allocation	Performance depends on time quantum

Suitability

- Emergency services → Priority Scheduling.
 - General services → Round Robin.
 - Mixed workloads → Priority-based Round Robin.
-

3. Impact on Emergency Patient Care

Poor scheduling may lead to:

- Delayed access to medical records.
- Slow diagnostic processing.
- Increased treatment delays.

Efficient scheduling provides:

- Faster emergency response.
 - Improved patient safety.
 - Better resource utilization.
 - Reduced waiting time for critical operations.
-

4. Most Appropriate Scheduling Algorithm

Priority-Based Round Robin Scheduling

Priority Levels

Service	Priority
Emergency Response	Highest
Medical Records	High
Diagnostic Services	Medium
Pharmacy Management	Medium
Patient Registration	Low

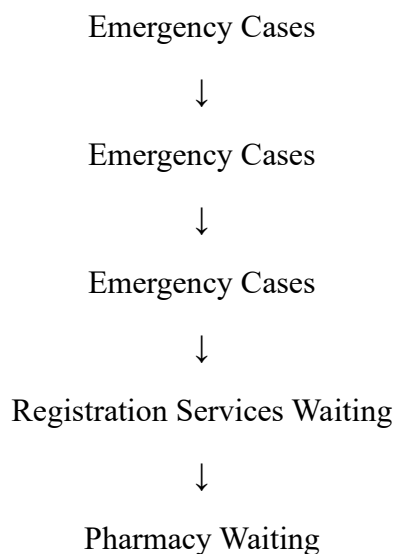
Emergency-related tasks should always receive higher priority.

5. Possibility of Starvation and Remedies

Starvation

Low-priority tasks may wait indefinitely if emergency requests continuously arrive.

Example



Remedy: Aging Technique

- Increase the priority of waiting processes over time.
- Long-waiting tasks are gradually promoted to higher-priority queues.

This prevents starvation.

6. Suitable Time Quantum for Round Robin

A suitable value is:

Time Quantum = 10–20 ms

Recommended:

15 ms

Benefits

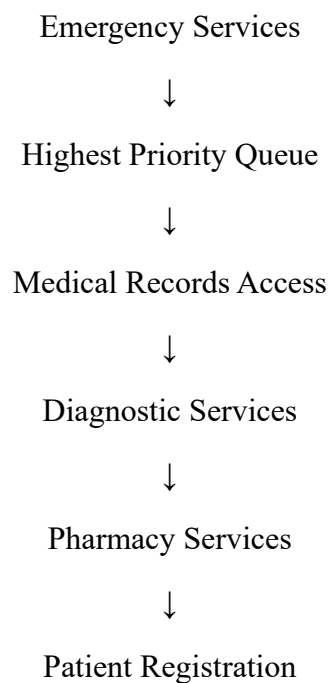
- Fast responsiveness.
 - Reduced waiting time.
 - Balanced context switching overhead.
 - Smooth execution of interactive hospital applications.
-

7. Performance During Mass Casualty Incidents

During peak situations:

- Emergency requests increase significantly.
- CPU utilization approaches maximum capacity.
- Background services may experience temporary delays.

Expected Behavior



The scheduling policy ensures critical services remain responsive.

8. Justification Using Performance Metrics

Metric	Improvement
CPU Utilization	High
Throughput	Improved
Response Time	Faster
Waiting Time	Reduced
Fairness	Better with Aging
Reliability	Improved

Priority-based scheduling significantly improves emergency response times while maintaining acceptable performance for routine tasks.

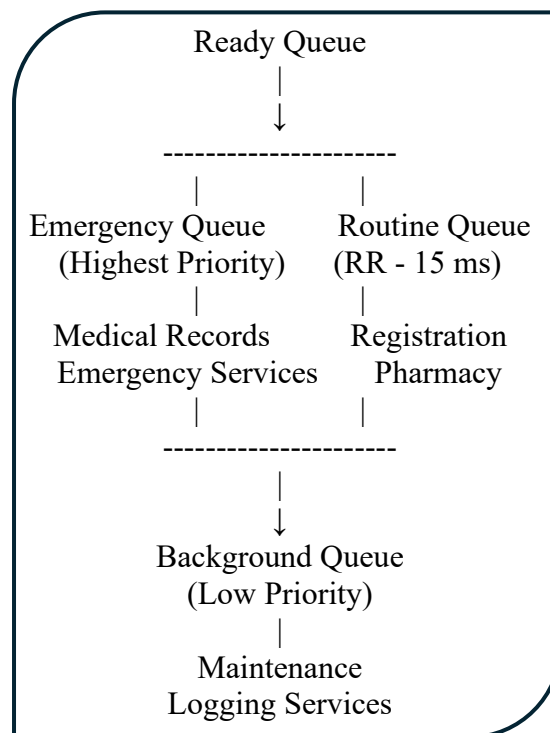
9. Proposed Modifications

1. Implement Aging to prevent starvation.
2. Use Multilevel Feedback Queues.
3. Separate emergency and routine service queues.
4. Introduce load balancing mechanisms.
5. Use thread pools for frequently accessed services.
6. Dynamically adjust priorities during emergencies.

These modifications improve:

- Fairness
 - Responsiveness
 - Scalability
 - Reliability
-

10. Scheduling Framework for Future Expansion



Framework Benefits

- Emergency services are always prioritized.
- Background processes execute without affecting critical tasks.

Conclusion

A **Priority-Based Round Robin Scheduling** approach with a **15 ms time quantum** and **aging mechanisms** provides the best balance between responsiveness, fairness, and CPU utilization. This framework ensures that emergency medical services receive immediate attention while routine hospital operations continue efficiently, making it well-suited for both current requirements and future expansion.

Question 9: Inter-Process Communication and Cooperating Processes

Scenario:

A smart city traffic management system integrates traffic signals, CCTV cameras, weather stations, public transportation services, and emergency vehicle tracking systems. These independent components continuously exchange information to optimize traffic flow and reduce congestion. However, communication failures between processes have occasionally resulted in delayed traffic signal updates and emergency response times. The city administration seeks an improved communication model to ensure uninterrupted coordination among all subsystems.

Questions:

1. Identify the cooperating processes involved in the smart city ecosystem.
2. Analyze the communication requirements among these processes.
3. Compare shared memory and message passing techniques for this environment.
4. Recommend the most suitable IPC mechanism and justify your choice.
5. Evaluate the impact of communication failures on public safety.
6. Propose a fault-tolerant IPC architecture for the city.
7. Predict potential challenges as the city expands its infrastructure.
8. Suggest methods to improve synchronization among cooperating processes.
9. Design a secure communication framework for the system.
10. Develop a scalability plan for future smart city implementations.

Solution:-

1. Cooperating Processes in the Smart City System

The following processes cooperate continuously:

Process	Function
Traffic Signal System	Controls traffic lights
CCTV Monitoring System	Detects traffic congestion and accidents
Weather Monitoring System	Provides weather updates

Process	Function
Public Transportation System	Tracks buses and metro services
Emergency Vehicle Tracking	Monitors ambulances and fire services
Traffic Analytics System	Performs traffic analysis
Notification System	Sends alerts and warnings

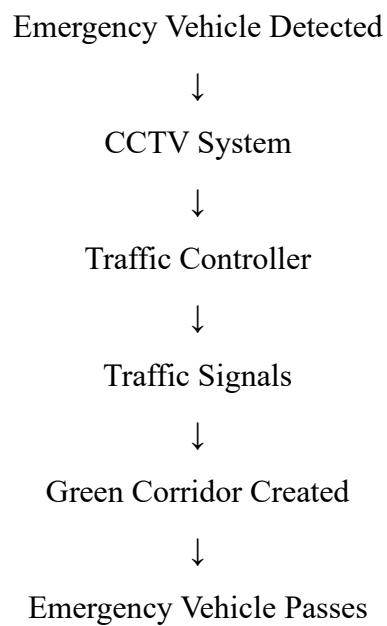
All these processes depend on one another for efficient traffic management.

2. Communication Requirements

The processes must communicate:

- Quickly
- Reliably
- Securely
- Continuously

Examples



Requirements include:

- Low latency
 - Real-time communication
 - Synchronization
 - Fault tolerance
 - Scalability
-

3. Shared Memory vs Message Passing

Feature	Shared Memory	Message Passing
Speed	Very High	High
Synchronization	Complex	Easier
Security	Lower	Better
Scalability	Moderate	High
Suitable for Distributed Systems	No	Yes
Fault Isolation	Low	High

Observation

Since smart city systems are geographically distributed, message passing is more suitable.

4. Recommended IPC Mechanism

Message Passing

Reasons

- Supports distributed systems.
- Easier synchronization.
- Better fault isolation.
- Highly scalable.
- Reliable communication among independent services.

Examples:

- Traffic Signals
- CCTV Servers
- Emergency Services
- Weather Stations

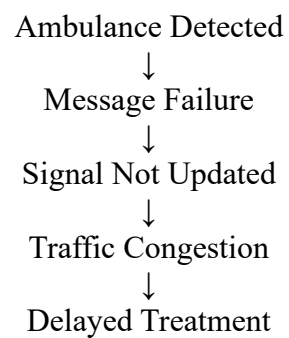
can communicate efficiently using message queues.

5. Impact of Communication Failures

Communication failures may result in:

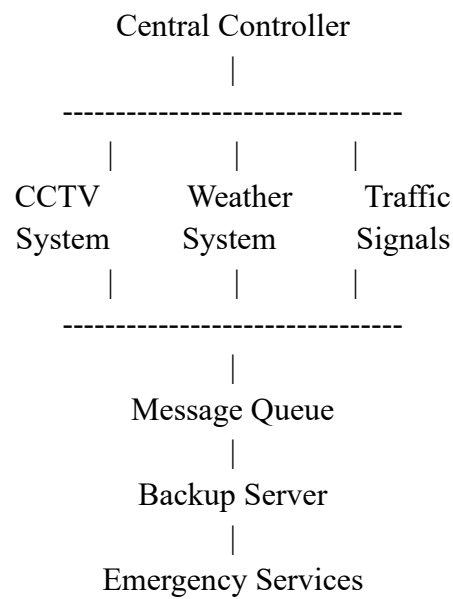
- Delayed emergency responses.
- Increased traffic congestion.
- Incorrect traffic signal timings.
- Reduced public safety.
- Failure to create emergency corridors.

Example



This can significantly affect public safety.

6. Fault-Tolerant IPC Architecture



Features

- Message buffering.
- Backup communication servers.
- Failure recovery mechanisms.
- Automatic retransmission of messages.

7. Future Challenges

As infrastructure expands:

- More processes will communicate simultaneously.
- Increased network traffic.
- Higher synchronization overhead.
- Greater security requirements.
- Larger data volumes from sensors and cameras.

Challenges include:

- Scalability.
- Performance bottlenecks.
- Resource management.

- Fault recovery.

8. Improving Synchronization

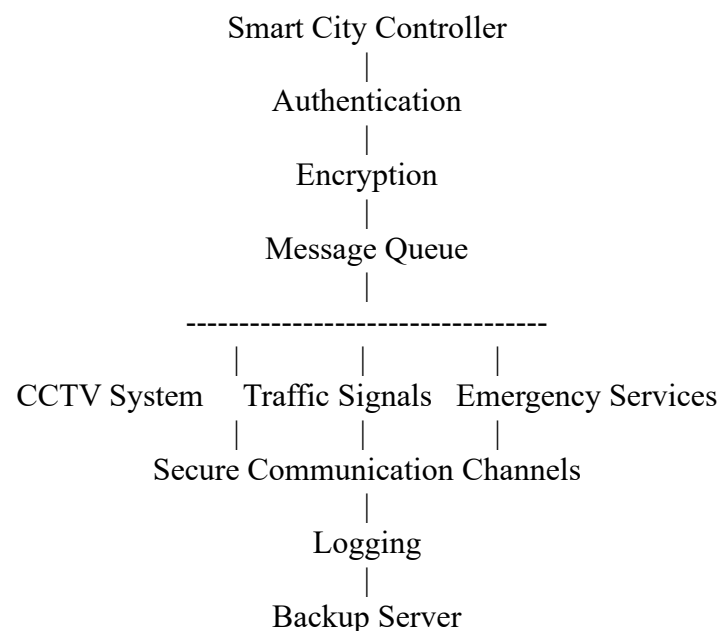
Methods include:

1. Semaphores.
2. Mutex locks.
3. Message queues.
4. Time synchronization mechanisms.
5. Event-driven communication models.
6. Load balancing techniques.

These improve:

- Consistency.
- Reliability.
- Process coordination.

9. Secure Communication Framework



Security Measures

- Authentication.
- Data encryption.

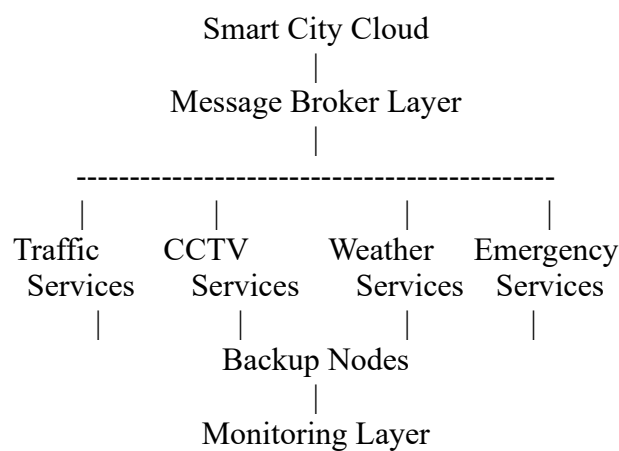
- Access control mechanisms.
- Secure message transmission.
- Continuous system monitoring.

10. Scalability Plan

Future implementations should include:

- Distributed message brokers.
- Cloud-based traffic management.
- Load balancing mechanisms.
- Automatic resource scaling.
- Backup communication channels.
- Microservice-based architectures.
- Redundant communication networks.

Scalable Architecture



This architecture supports millions of communications efficiently.

Conclusion

A **Message Passing IPC mechanism** is the most suitable choice for a smart city traffic management system because it provides scalability, fault tolerance, and secure communication across distributed components. By incorporating message queues, backup servers, synchronization mechanisms, and cloud-based scalability, the system can ensure reliable coordination among traffic services while maintaining public safety and supporting future expansion.

Question 10: Threads and Multithreading Models

Scenario:

A global media company streams live sporting events to millions of viewers worldwide. To improve user experience, the application uses separate threads for video decoding, audio synchronization, subtitle generation, buffering, and advertisement insertion. During high-demand events such as the FIFA World Cup final, users occasionally experience video freezing and synchronization issues. The software architects have been asked to investigate whether the current multithreading model is appropriate for handling massive workloads.

Questions:

1. Identify the threads involved in the streaming application.
2. Compare user-level and kernel-level threads for this environment.
3. Analyze the suitability of One-to-One, Many-to-One, and Many-to-Many models.
4. Evaluate the consequences of thread blocking during live broadcasts.
5. Recommend an appropriate multithreading model with justification.
6. Predict system behavior under a tenfold increase in user traffic.
7. Suggest techniques to improve thread synchronization.
8. Design a thread management strategy for uninterrupted streaming.
9. Evaluate the role of multithreading in enhancing application performance.
10. Develop a monitoring framework for detecting thread-related issues.

Solution:-

1. Threads Involved in the Streaming Application

The media streaming platform uses multiple threads simultaneously.

Thread	Function
Video Decoding Thread	Decodes video frames
Audio Synchronization Thread	Synchronizes audio with video
Subtitle Thread	Displays subtitles
Buffer Management Thread	Handles buffering operations
Advertisement Thread	Inserts advertisements

Thread	Function
Network Thread	Receives streaming packets
User Interface Thread	Handles user interactions
Monitoring Thread	Tracks performance and errors

These threads work together to provide uninterrupted live streaming.

2. User-Level vs Kernel-Level Threads

Feature	User-Level Threads	Kernel-Level Threads
Management	User Space	Operating System
Context Switching	Faster	Slightly slower
Parallel Execution	Limited	Supported
Blocking Issue	Entire process may block	Only blocked thread waits
Multiprocessor Support	Limited	Excellent
Suitable for Streaming	Moderate	High

Kernel-Level Threads are more suitable because they support true parallelism and better fault isolation.

3. Comparison of Multithreading Models

Model	Characteristics	Suitability
Many-to-One	Multiple user threads mapped to one kernel thread	Poor
One-to-One	Each user thread has one kernel thread	Excellent
Many-to-Many	Multiple user threads mapped to multiple kernel threads	Excellent for scalability

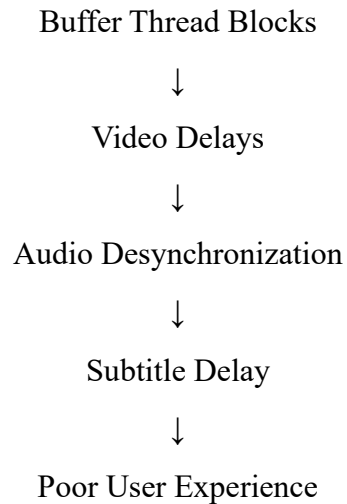
Analysis

- **Many-to-One:** If one thread blocks, the entire application may stop.
- **One-to-One:** Provides better responsiveness and parallel execution.

- **Many-to-Many:** Offers scalability and efficient resource utilization for large workloads.
-

4. Consequences of Thread Blocking

If the buffering thread blocks:



Possible consequences include:

- Video freezing.
 - Audio synchronization issues.
 - Increased buffering time.
 - Advertisement insertion failures.
 - Reduced viewer satisfaction.
-

5. Recommended Multithreading Model

Many-to-Many Multithreading Model

Justification

- Highly scalable.
 - Supports millions of simultaneous users.
 - Better load distribution.
 - Efficient utilization of multicore processors.
 - Prevents a single blocked thread from affecting the entire application.
-

6. System Behavior Under Tenfold User Traffic

With ten times more users:

Without Proper Scaling

- Increased buffering delays.
- Higher CPU utilization.
- Thread contention.
- Network congestion.
- Reduced throughput.

With Many-to-Many Model

- Dynamic thread allocation.
- Improved load balancing.
- Better responsiveness.
- Higher scalability.

The system remains more stable during large sporting events.

7. Techniques for Improving Thread Synchronization

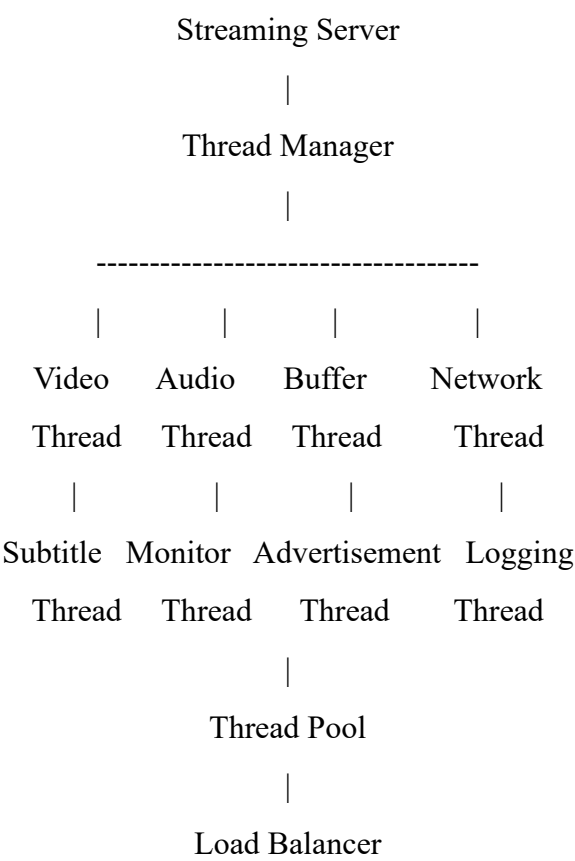
The following techniques can improve synchronization:

- Mutex locks.
- Semaphores.
- Condition variables.
- Read-write locks.
- Thread pools.
- Lock-free data structures.
- Producer-consumer synchronization mechanisms.

These techniques help avoid:

- Race conditions.
 - Deadlocks.
 - Data inconsistencies.
-

8. Thread Management Strategy



Features

- Dynamic thread allocation.
- Automatic load balancing.
- Failure recovery mechanisms.
- Efficient CPU utilization.

9. Role of Multithreading in Performance

Multithreading provides:

- Higher throughput.
- Improved responsiveness.
- Better CPU utilization.
- Reduced latency.
- Efficient resource sharing.
- Parallel execution on multicore processors.

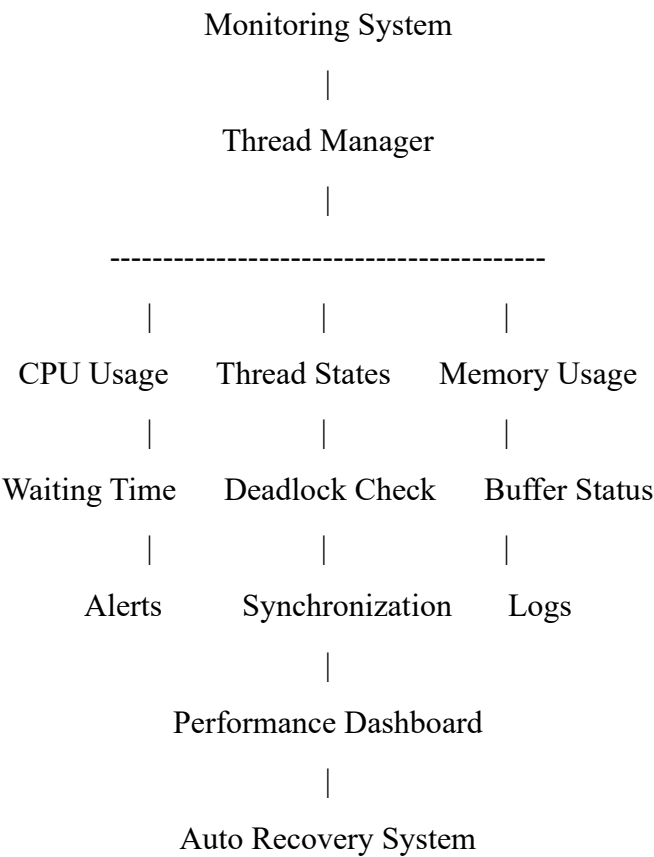
Example

While:

- Video is being decoded,
- Audio is synchronized,
- Subtitles are generated,
- Buffering occurs simultaneously.

This significantly improves user experience.

10. Monitoring Framework



Monitoring Metrics

- CPU utilization.
- Thread waiting time.
- Buffer occupancy.
- Synchronization failures.
- Deadlock detection.

- Response time.
- Throughput analysis.

The framework enables proactive detection and recovery of thread-related issues.

Conclusion

A **Many-to-Many multithreading model** is the most suitable choice for large-scale live streaming applications because it provides scalability, fault isolation, and efficient resource utilization. Combined with kernel-level threads, synchronization mechanisms, dynamic thread management, and continuous monitoring, it ensures uninterrupted streaming and optimal performance even during high-demand events such as the FIFA World Cup final.
